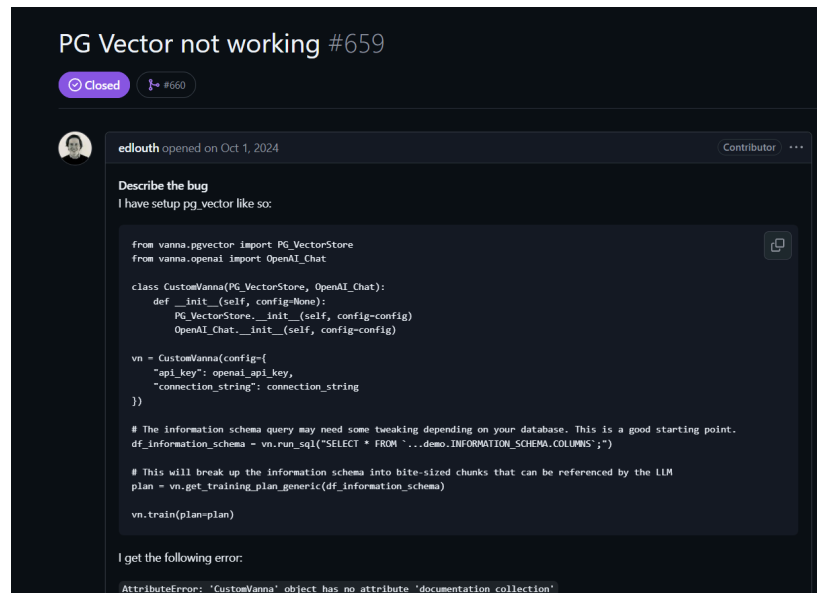


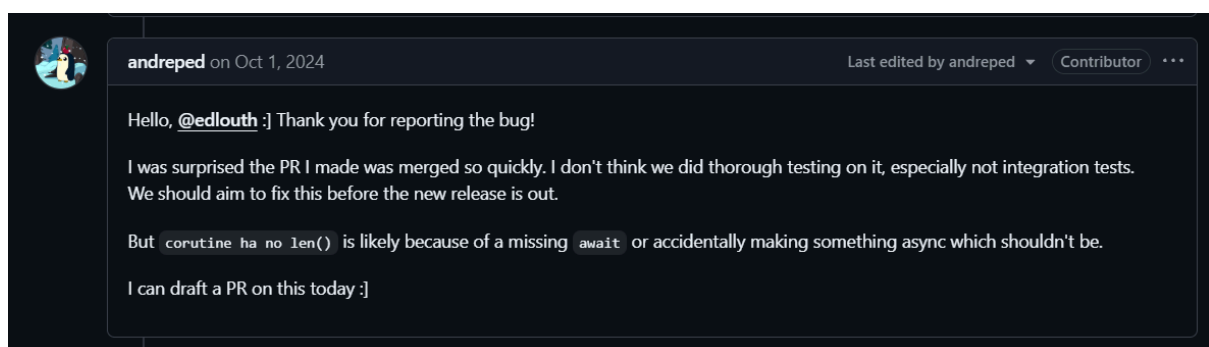
Eixo A.1 — Arqueologia de Issues

Issue analisada: #659 — PG Vector not working

Pull Request relacionado: #660 — Pgvector fixes



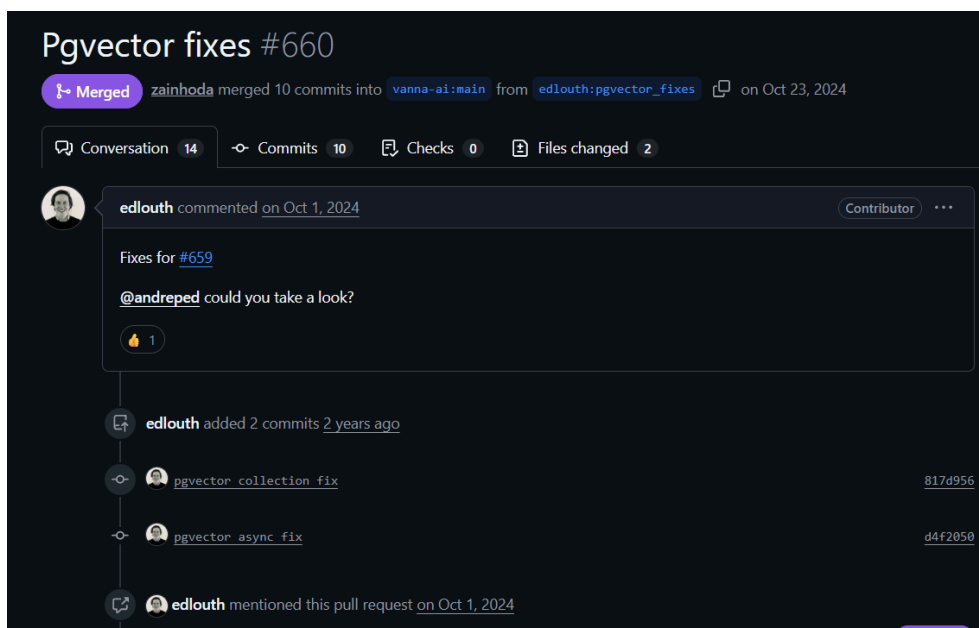
A issue analisada foi aberta pelo usuário edlouth, que relatou falhas na integração com o PGVector dentro do projeto Vanna AI. Entre os principais problemas identificados estavam erros relacionados ao atributo inexistente `documentation_collection`, além de falhas envolvendo execução assíncrona, como o erro `coroutine has no len()`. O relato também demonstrava dificuldades na utilização prática do PGVector em conjunto com embeddings e armazenamento vetorial.



Durante a investigação, observou-se intensa colaboração entre os contribuidores do projeto. Logo nos primeiros comentários, um dos desenvolvedores reconheceu que a funcionalidade havia sido integrada sem testes adequados, afirmando que o processo de validação não havia sido suficientemente aprofundado antes do merge inicial da implementação. Essa evidência demonstra fragilidade no processo de

qualidade do software e indica possível pressão por entrega rápida da funcionalidade.

Ao longo da discussão, diferentes hipóteses técnicas foram levantadas para explicar os problemas encontrados. Entre elas, destacam-se inconsistências relacionadas ao uso simultâneo de métodos síncronos e assíncronos, problemas na implementação da função de embeddings e dificuldades de compatibilidade com a arquitetura do PGVector. A discussão evoluiu de forma colaborativa, envolvendo análise técnica do código, revisão entre desenvolvedores e refinamento gradual das soluções propostas.



Como resposta aos problemas identificados, foi criada a Pull Request #660, responsável por consolidar as correções da funcionalidade. A análise da PR demonstra um fluxo mais estruturado de validação, incluindo revisão colaborativa do código, novos commits corretivos, ajustes na implementação dos embeddings e execução de testes automatizados. Também foram identificadas discussões relacionadas à execução dos testes do projeto utilizando o comando:

```
pytest -v tests/test_pgvector.py
```

Os comentários da Pull Request mostram que os desenvolvedores realizaram validações adicionais antes da integração final da solução, demonstrando preocupação posterior com estabilidade e qualidade da funcionalidade.

As alterações implementadas no código envolveram a correção da integração com o PGVector, a padronização entre métodos síncronos e assíncronos, a remoção de métodos redundantes e melhorias na implementação da função de embeddings. Além disso, os testes automatizados foram atualizados para validar corretamente o comportamento da funcionalidade após as correções. Essas mudanças contribuíram

diretamente para reduzir falhas de execução e aumentar a estabilidade da integração vetorial do sistema.

Outro ponto relevante observado durante a análise foi a mudança gradual de escopo da issue. Inicialmente, o foco da discussão estava concentrado em um erro específico relacionado ao funcionamento do PGVector. Entretanto, durante o processo de investigação, foram identificados outros problemas arquiteturais e de qualidade, como falhas nos testes, inconsistências no uso de programação assíncrona e limitações na implementação dos embeddings. Dessa forma, a correção deixou de tratar apenas um erro isolado e passou a envolver melhorias mais amplas na arquitetura da funcionalidade.

A análise geral demonstra que o projeto possui boa rastreabilidade técnica, permitindo acompanhar de forma clara o ciclo completo da funcionalidade, desde a abertura da issue até o merge da Pull Request e posterior lançamento de uma nova versão do sistema. Também foi possível observar forte colaboração entre contribuidores e rápida capacidade de resposta aos problemas relatados pela comunidade.

Entretanto, a investigação evidencia fragilidades importantes no processo de qualidade do software, especialmente relacionadas à ausência inicial de testes adequados antes da integração da funcionalidade. Além disso, observou-se que parte significativa das decisões técnicas ocorre diretamente nos comentários das issues e Pull Requests, **sem utilização de processos formais de documentação ou RFCs (Request for Comments)**. Embora essa abordagem torne o desenvolvimento mais ágil, ela reduz a formalização da engenharia de requisitos e pode dificultar a rastreabilidade de decisões arquiteturais mais complexas ao longo da evolução do projeto.

Com base na análise realizada, **a funcionalidade foi classificada com risco técnico de nível médio-alto**. Essa classificação se justifica pelo fato de que os problemas identificados afetavam diretamente componentes críticos do sistema, como armazenamento vetorial, execução de consultas e integração com embeddings. Além disso, foram identificadas inconsistências arquiteturais envolvendo programação assíncrona e ausência inicial de testes robustos. Apesar disso, o projeto demonstrou boa capacidade de correção colaborativa, revisão técnica e validação posterior das soluções implementadas.

Diante desse cenário, recomenda-se o fortalecimento do processo de testes antes do merge de novas funcionalidades, a adoção de validações automatizadas obrigatórias em Pull Requests, maior padronização entre componentes síncronos e assíncronos e uma formalização mais clara do processo de mudanças técnicas. Essas ações contribuiriam para aumentar a maturidade do processo de desenvolvimento e reduzir riscos arquiteturais futuros.

Eixo A.2 — Gestão de Riscos Ocultos (MPS.BR – GPR)

Gestão de Riscos Ocultos consiste no processo de identificação, análise e mitigação de problemas técnicos que não estão necessariamente documentados formalmente, mas que podem comprometer a estabilidade, segurança, manutenção ou evolução do software. Esses riscos geralmente aparecem de forma indireta no código-fonte, em comentários temporários, dependências frágeis, ausência de tratamento de falhas, acoplamento excessivo ou soluções improvisadas.

No contexto da análise do projeto Vanna AI, a investigação foi realizada principalmente por meio da inspeção da arquitetura modular e dos mecanismos internos relacionados à integração com APIs de Inteligência Artificial.

Na pasta **integrations/**, o projeto apresenta módulos independentes para diferentes provedores e serviços externos, como:

vanna / src / vanna / integrations /			↑ Top
anthropic	apply ruff formatting	6 months ago	
azureopenai	Run ruff formatting	6 months ago	
azuresearch	apply ruff formatting	6 months ago	
bigquery	apply ruff formatting	6 months ago	
chromadb	Fix ruff formatting errors in ChromaDB files	3 months ago	
clickhouse	apply ruff formatting	6 months ago	
duckdb	apply ruff formatting	6 months ago	
faiss	Fix for text memory retrieval issue	6 months ago	
google	update chroma embedding model param	6 months ago	
hive	apply ruff formatting	6 months ago	
local	apply ruff formatting	6 months ago	
marqo	apply ruff formatting	6 months ago	
milvus	apply ruff formatting	6 months ago	
mock	begin refactor for vanna2	7 months ago	
mssql	apply ruff formatting	6 months ago	
mysql	apply ruff formatting	6 months ago	

Essa organização demonstra preocupação com o desacoplamento da arquitetura e facilita substituições futuras de provedores de IA, mitigando o risco de vendor-lock-in, alinhando-se ao conceito de mitigação de risco arquitetural recomendado pelo MPS.BR e pelas boas práticas de engenharia de software. **(RISCO BAIXO/MÉDIO)**

No módulo **integrations/**, em cada pasta de provedor foi identificado o uso de variáveis de ambiente para seleção dinâmica do modelo, logo, a versão pode ser alterada sem alteração do código, reduzindo, assim, o risco de mudança de versões **(RISCO BAIXO)**

```
self.model = model or os.getenv("OPENAI_MODEL", "gpt-5")
```

Além disso, o sistema centraliza a comunicação com provedores de IA em classes especializadas que implementam uma abstração comum, caracterizando uma abordagem compatível com o padrão Strategy. Essa arquitetura favorece baixo acoplamento, intercambialidade entre provedores e maior flexibilidade para adaptação a mudanças nas APIs externas, assim, colaborando com a mitigação do risco do vendor lock in

```
class OpenAIResponsesService(LlmService):
    def __init__(
        self, api_key: Optional[str] = None, model: Optional[str] = None
    ) -> None:
```

Também foi identificado o módulo **core/recovery**, responsável por estratégias de recuperação de falhas, sua implementação garante que o sistema não simplesmente quebra quando ocorre timeout, falhas de API ou indisponibilidade do modelo;

```
class ErrorRecoveryStrategy(ABC):
    """Strategy for handling errors and implementing retry logic.

    |
    Subclass this to create custom error recovery strategies that can:
    - Retry failed operations with backoff
    - Fallback to alternative approaches
    - Log errors to external systems
    - Gracefully degrade functionality
```

Ele possui uma camada dedicada para decidir: se deve tentar novamente, esperar, falhar ou usar fallback, logo, é importante para a mitigação de riscos de instabilidade ao utilizar um provedor de IA. **(RISCO MÉDIO)**

No módulo **core/filter**, foram encontrados mecanismos voltados ao controle de contexto e filtragem de mensagens antes do envio aos modelos, realizando remoção de informações sensíveis, controle de limite de contexto, de duplicação de mensagem e priorização de mensagens relevantes.

```
class ConversationFilter(ABC):  
    """Filter for transforming conversation history.  
  
    Subclass this to create custom filters that can:  
    - Remove sensitive information  
    - Summarize long conversations  
    - Manage context window limits  
    - Deduplicate similar messages  
    - Prioritize recent or relevant messages
```

Garantindo, assim, uma mitigação do risco de alucinações da IA, ou seja, quando é retornada uma resposta sem relação com o prompt e uma redução de custos ao só enviar para a IA o que é essencial para compreensão do prompt. **(RISCO BAIXO/MÉDIO)**

Também foi realizada uma inspeção textual no repositório em busca de marcações normalmente associadas a débitos técnicos ou riscos negligenciados, como: TODO, FIXME e HACK, porém não foram identificadas ocorrências significativas dessas marcações, nem grandes blocos de código comentado abandonado, o que sugere que o projeto mantém um nível razoável de organização e manutenção contínua do código-fonte. Entretanto, a ausência dessas marcações também pode dificultar a rastreabilidade explícita de débitos técnicos e pendências internas do projeto. **(RISCO MÉDIO)**

Conclusão

A análise evidencia que o projeto apresenta preocupação significativa com riscos técnicos associados ao uso de Inteligência Artificial, especialmente em relação à volatilidade de APIs externas, modularização de provedores de IA e mecanismos de validação estrutural do sistema.

Observou-se a adoção de estratégias arquiteturais voltadas à mitigação de falhas operacionais, como separação de integrações, mecanismos de recuperação de erros e validações automatizadas, contribuindo para maior robustez e manutenibilidade da aplicação.

Dessa forma, conclui-se que o projeto demonstra um nível intermediário de maturidade técnica no gerenciamento de riscos relacionados à IA, apresentando preocupação consistente com qualidade arquitetural. Entretanto, ainda foram identificadas lacunas relacionadas à governança operacional, ou seja, um processo de gestão de risco padronizado, observabilidade para uma maior robustez na monitoração, estratégias formais de fallback relacionados a

ações corretivas e documentação explícita do tratamento de riscos técnicos, aspectos recomendados por modelos de maturidade como MPS.BR e CMMI.

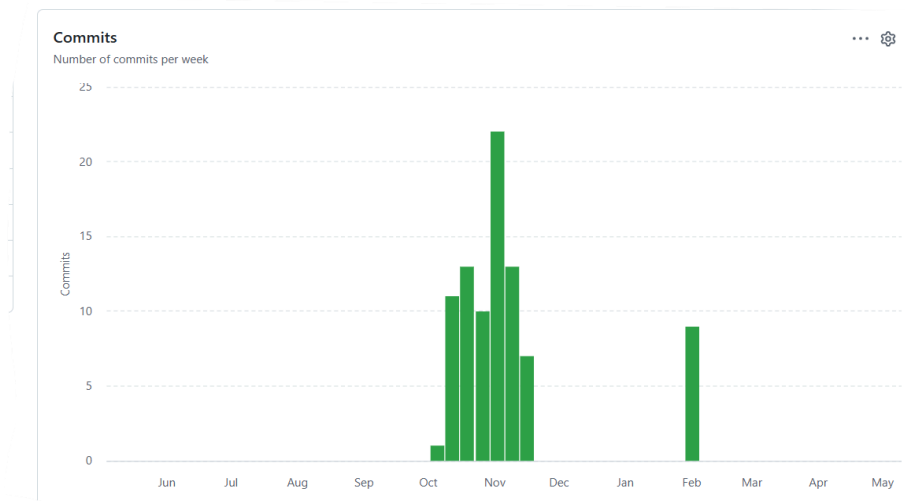
A.3 — Ritmo de Entrega e Code Review

Ritmo de Entrega corresponde à forma como as contribuições e atualizações de um projeto são distribuídas ao longo do tempo, permitindo analisar se o desenvolvimento ocorre de maneira contínua e organizada ou em ciclos concentrados de alta atividade (“crunch”). Essa análise auxilia na identificação de maturidade gerencial, previsibilidade do desenvolvimento e estabilidade do processo de manutenção do software.

Já o Code Review consiste no processo de revisão técnica de alterações no código-fonte antes de sua integração ao projeto principal. Essa prática tem como objetivo identificar defeitos, validar decisões arquiteturais e garantir aderência aos padrões do projeto.

No contexto da análise do projeto Vanna AI, o Ritmo de Entrega foi investigado por meio da análise do gráfico de contribuições/commits e do histórico de Pull Requests no GitHub.

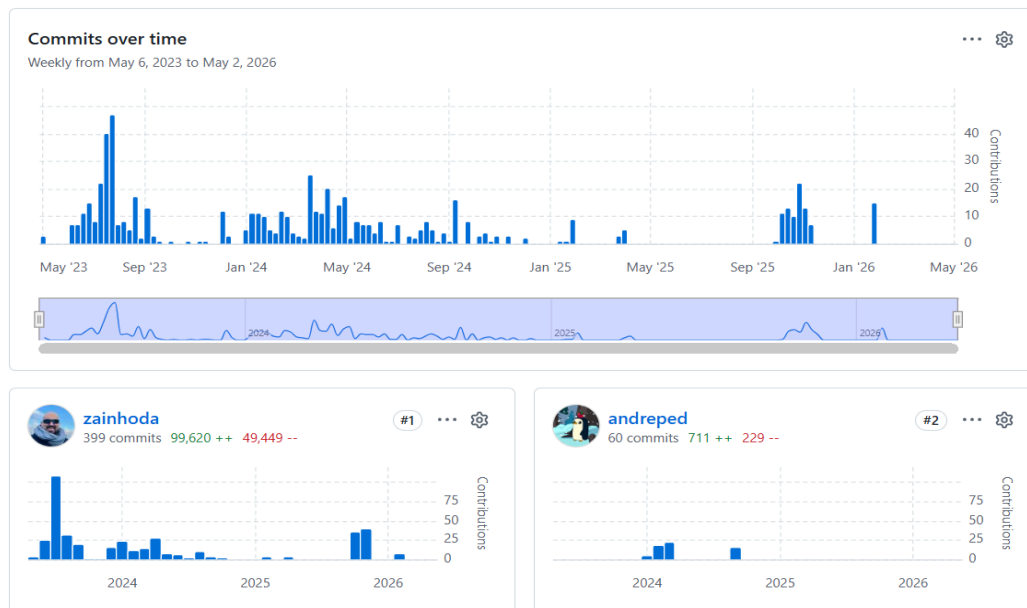
A análise do gráfico de contribuições do repositório demonstra que o projeto apresenta períodos de alta atividade intercalados com longos intervalos de baixa movimentação. Os gráficos de commits evidenciam picos concentrados principalmente entre os meses de agosto de 2023, março de 2024 e novembro de 2025, seguidos por períodos relativamente extensos com poucas contribuições, caracterizando o projeto por vários picos de “crunch”, aumentando riscos relacionados à introdução de débitos técnicos, redução da previsibilidade do processo de desenvolvimento, sobrecarga dos mantenedores e diminuição da qualidade das revisões de código. **(RISCO MÉDIO/ALTO)**



Esse comportamento sugere um modelo de manutenção predominantemente reativo, no qual grandes volumes de commits ocorrem concentrados em momentos específicos, normalmente associados à correção de bugs, melhorias arquiteturais ou ajustes relacionados às integrações com modelos de IA e bancos vetoriais.

Também foi identificado um forte desbalanceamento entre os contribuidores do projeto, já que o principal desenvolvedor possui aproximadamente seis vezes mais commits que o segundo maior contribuidor do repositório, aumentando o risco de haver um “herói”, termo utilizado na Engenharia de Software para descrever cenários em que uma única pessoa concentra grande parte do conhecimento técnico, da manutenção e das decisões arquiteturais do sistema. Esse cenário pode gerar risco de dependência excessiva de um único mantenedor, dificultando a escalabilidade da equipe, continuidade do projeto e transferência de conhecimento, pois caso o principal desenvolvedor deixe o projeto, parte significativa do

conhecimento arquitetural e operacional pode ser perdida(**RISCO ALTO**)



Durante a análise dos Pull Requests do projeto Vanna AI, foram identificadas evidências da utilização de práticas de Code Review no processo de desenvolvimento.

Os Pull Requests analisados apresentavam descrição detalhada do problema; justificativa técnica das alterações; documentação dos impactos da correção; checklist de validação; referência à issues e vulnerabilidades; solicitações formais de revisão e aprovações explícitas de outros contribuidores.

Exemplo de um Pull Request: <https://github.com/vanna-ai/vanna/issues/1121>

Esses elementos demonstram que o projeto adota mecanismos básicos de revisão colaborativa antes da integração de alterações ao código principal, indicando preocupação com qualidade e rastreabilidade das mudanças realizadas.

Entretanto, apesar da existência de aprovações formais, observou-se baixa quantidade de discussões técnicas aprofundadas nos comentários dos Pull Requests analisados. Em vários casos, os PRs foram aprovados com poucas interações entre revisores, sem debates arquiteturais extensos ou questionamentos técnicos mais detalhados, o que indica um processo de revisão mais focado em validação funcional imediata e integração rápida das alterações. (**RISCO MÉDIO**)

Dessa forma, conclui-se que o projeto apresenta evidências reais de revisão de código, porém ainda com limitações relacionadas à profundidade colaborativa das análises técnicas realizadas durante o processo de aprovação dos Pull Requests.

Conclusão

A análise do eixo A.3 evidencia que o projeto Vanna AI apresenta práticas reais de manutenção colaborativa e revisão de código, demonstrando preocupação com qualidade técnica, rastreabilidade das alterações e correção de falhas relevantes. Entretanto, o ritmo de desenvolvimento mostrou-se irregular, caracterizado por ciclos de “crunch” e forte concentração de contribuições em poucos desenvolvedores, aumentando riscos relacionados à dependência de mantenedores específicos, débitos técnicos e redução da previsibilidade do processo de evolução do sistema.

Além disso, embora existam evidências formais de Code Review, observou-se baixa profundidade nas discussões técnicas realizadas nos Pull Requests, indicando um processo de revisão mais orientado à validação funcional imediata do que à análise arquitetural aprofundada. Dessa forma, conclui-se que o projeto apresenta um nível intermediário de maturidade gerencial e colaborativa, ainda possuindo oportunidades de evolução em governança de desenvolvimento, ou seja, na forma como o projeto se organiza, distribuição de conhecimento e fortalecimento das práticas de revisão técnica recomendadas por modelos como MPS.BR e CMMI.

Eixo B.1 - O Teste da Troca

Se fosse necessário trocar o provedor de inteligência artificial do sistema, por exemplo substituindo a OpenAI por um modelo local como o Llama-3, o impacto no código dependeria diretamente do nível de desacoplamento da arquitetura. A partir da análise do código do Agent, é possível observar que ele não depende diretamente de nenhuma implementação específica de modelo de linguagem, mas sim de uma abstração chamada LlmService. Essa abstração é injetada no agente no momento da sua criação, o que indica que o sistema foi projetado seguindo o Princípio da Inversão de Dependência (DIP), pois o módulo principal depende de uma interface e não de uma implementação concreta.

```
vanna / src / vanna / core / agent / agent.py
zainhoda more component pruning 5542b3b · 6 months ago History
Code Blame 1407 lines (1227 loc) · 60.2 KB
1 """
2 Agent implementation for the Vanna Agents framework.
3
4 This module provides the main Agent class that orchestrates the interaction
5 between LLM services, tools, and conversation storage.
6 """
7
8 import traceback
9 import uuid
10 from typing import TYPE_CHECKING, AsyncGenerator, List, Optional
11
12 from vanna.components import (
13     UIComponent,
14     SimpleTextComponent,
15     RichTextComponent,
16     StatusBarUpdateComponent,
17     TaskTrackerUpdateComponent,
18     ChatInputUpdateComponent,
19     StatusCardComponent,
20     Task,
21 )
```

Na prática, isso significa que a responsabilidade de comunicação com o modelo de linguagem não está dentro do Agent, mas sim em implementações externas localizadas na camada de integrações do sistema. Essas implementações podem variar, como serviços baseados em OpenAI, Anthropic ou modelos locais, desde que respeitem o contrato definido por LlmService. Dessa forma, a troca de provedor de IA ocorreria apenas nessa camada de integração, sem necessidade de alterações no núcleo do agente.

Name	Last commit message	Last commit date
..		
anthropic	apply ruff formatting	6 months ago
azureopenai	Run ruff formatting	6 months ago
azuresearch	apply ruff formatting	6 months ago
bigquery	apply ruff formatting	6 months ago
chromadb	Fix ruff formatting errors in ChromaDB files	3 months ago
clickhouse	apply ruff formatting	6 months ago
duckdb	apply ruff formatting	6 months ago
faiss	Fix for text memory retrieval issue	6 months ago
google	update chroma embedding model param	6 months ago
hive	apply ruff formatting	6 months ago

Em um cenário ideal, a substituição do provedor exigiria a modificação de apenas um ou dois arquivos, geralmente relacionados à implementação específica do serviço de LLM e, possivelmente, a algum arquivo de configuração responsável por definir qual provedor será utilizado. No entanto, caso o sistema não estivesse corretamente desacoplado, essa troca poderia impactar múltiplos arquivos, incluindo

o próprio agente e outras partes do código que fariam referência direta ao provedor específico, o que caracterizaria uma violação do Princípio da Inversão de Dependência.

Portanto, com base na estrutura analisada, o sistema demonstra um bom nível de abstração e modularidade, permitindo a troca de provedores de IA com impacto mínimo no código e mantendo o núcleo da aplicação independente de implementações concretas.

Eixo B.2 - Busca por "God Objects"

A classe com a maior quantidade de linhas é a VannaBase contanto com 2125 linhas e 65 métodos. Sendo encontrada em:

<https://github.com/vanna-ai/vanna/blob/main/src/vanna/legacy/base/base.py>

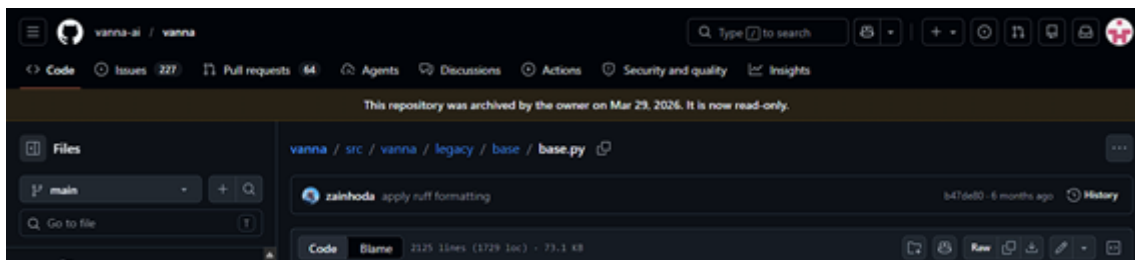


Figura 1: Local da classe VannaBase

Para descobrir a classe com a maior quantidade de linhas utilizou o seguinte procedimento:

```
PS C:\Users\jseol\OneDrive\Área de Trabalho\Desenvolvimento> cd .\vanna\
PS C:\Users\jseol\OneDrive\Área de Trabalho\Desenvolvimento\vanna> Get-Childitem -Recurse -Filter *.py |
>> Select-Object FullName,
>> @{Name="Lines";Expression=((Get-Content $_.FullName).Count)} |
>> Sort-Object Lines -Descending |
>> Select-Object -First 20

FullName                                                                 Lines
-----
C:\Users\jseol\OneDrive\Área de Trabalho\Desenvolvimento\vanna\src\vanna\legacy\base\base.py      2125
C:\Users\jseol\OneDrive\Área de Trabalho\Desenvolvimento\vanna\src\vanna\core\agent\agent.py      1407
C:\Users\jseol\OneDrive\Área de Trabalho\Desenvolvimento\vanna\src\vanna\legacy\flask\_init_.py    1343
C:\Users\jseol\OneDrive\Área de Trabalho\Desenvolvimento\vanna\tests\test_tool_permissions.py     915
```

Figura 2: Método para descobrir maior classe

É uma classe que apresenta diversos métodos caracterizando uma God Class, alguns dos métodos encontrado nela:

- `__init__(config)` • `generate_sql(question, ...)` • `extract_sql(llm_response)` • `is_sql_valid(sql)` • `should_generate_chart(df)` • `generate_rewritten_question(...)` • `generate_followup_questions(...)` • `generate_summary(question, df)` • `generate_embedding(data)` • `get_similar_question_sql(question)` • `get_related_ddl(question)` • `add_question_sql(question, sql)` • `add_ddl(ddl)` • `run_sql(property setter/getter)` • `connect_to_snowflake(...)` • `connect_to_sqlite(url)` •

connect_to_postgres(...) • connect_to_mysql(...) • connect_to_oracle(...) • connect_to_bigquery(...) [... + 45 mais métodos ...]

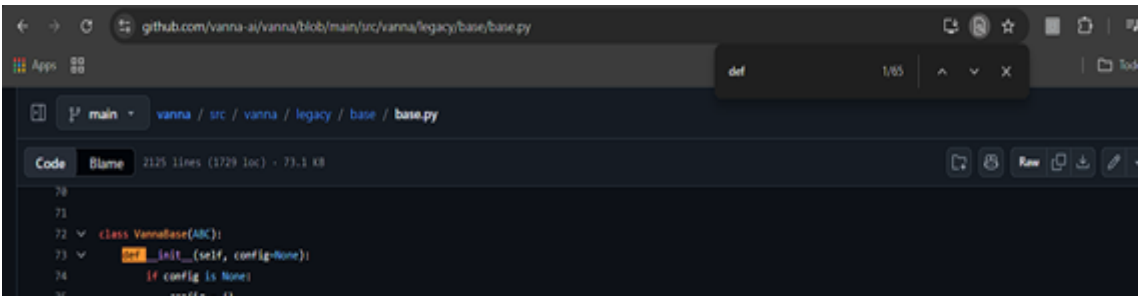


Figura 3: Quantidade de métodos

A VannaBase viola massivamente o Princípio da Responsabilidade Única (SRP). A classe mescla 7 responsabilidades distintas:

Geração de SQL	generate_sql() extract_sql()
Validação e processamento de SQL	is_sql_valid() get_sql_prompt()
Gerenciamento de conexões de banco de dados	10 métodos connect_to_*
Execução de SQL	run_sql (setter via closures)
Recuperação e treinamento de embeddings	generate_embedding() add_question_sql() add_ddl() add_documentation()
Orquestração de fluxo	ask() train() generate_summary()
Gerenciamento de LLM e prompts	submit_prompt() get_sql_prompt() system_message() user_message() assistant_message()

Recomenda-se dividir a classe em componentes especializados:

Classe proposta	Responsabilidade
-----------------	------------------

SqlGenerationService	Responsável por generate_sql(), extract_sql(), is_sql_valid()
DatabaseConnectorFactory	Centraliza métodos connect_to_* e gerenciamento de conexões
EmbeddingService	Gerencia embeddings e treinamento (generate_embedding(), add_ddl(), add_documentation())
PromptManager	Controle de prompts e mensagens (system_message(), user_message(), assistant_message())
WorkflowOrchestrator	Coordena o fluxo (ask(), train(), generate_summary())

Exemplo conceitual:

```
class SqlGenerationService:
    def generate_sql(self, question): ...
    def extract_sql(self, response): ...
    def is_sql_valid(self, sql): ...

class EmbeddingService:
    def generate_embedding(self, data): ...
    def add_ddl(self, ddl): ...

class PromptManager:
    def system_message(self): ...
    def user_message(self): ...

class WorkflowOrchestrator:
    def __init__( ...
```

Eixo B.3 - Métricas de Duplicação (DRY)

B.3 solicita a identificação de padrões de código repetidos, como tratamento de erros de API, configuração de headers ou outras lógicas que poderiam ter sido abstraídas. No caso do Vanna, a análise mostra uma duplicação parcial em partes responsáveis por integrar diferentes provedores de LLM, especialmente nas classes de integração com OpenAI, Anthropic e Google Gemini.

A primeira evidência está na forma como os provedores de LLM são implementados. O projeto define uma interface abstrata chamada “LlmService”, que obriga os provedores a implementarem métodos como os da imagem abaixo. Isso é positivo arquiteturalmente, pois padroniza o contrato de comunicação com diferentes modelos, mas, ao observar as implementações concretas, percebe-se que cada provedor repete partes semelhantes do fluxo que vai de leitura de variáveis de ambiente e criação do cliente externo até interpretação da resposta e retorno em um modelo comum.

```
vanna / src / vanna / core / llm / base.py
zainhoda clean up mypy issues

Code Blame 40 lines (30 loc) · 1.04 KB

1  """
2  LLM domain interface.
3
4  This module contains the abstract base class for LLM services.
5  """
6
7  from abc import ABC, abstractmethod
8  from typing import Any, AsyncGenerator, List
9
10 from .models import LlmRequest, LlmResponse, LlmStreamChunk
11
12
13 class LlmService(ABC):
14     """Service for LLM communication."""
15
16     @abstractmethod
17     async def send_request(self, request: LlmRequest) -> LlmResponse:
18         """Send a request to the LLM."""
19         pass
20
21     @abstractmethod
22     async def stream_request(
23         self, request: LlmRequest
24     ) -> AsyncGenerator[LlmStreamChunk, None]:
25         """Stream a request to the LLM.
26
27         Args:
28             request: The LLM request to stream
29
30         Yields:
31             LlmStreamChunk instances as they arrive
32         """
33         # This is an async generator method
34         raise NotImplementedError
35         yield # pragma: no cover - makes this an async generator
36
37     @abstractmethod
38     async def validate_tools(self, tools: List[Any]) -> List[str]:
39         """Validate tool schemas and return any errors."""
40         pass
```

Um exemplo mais claro de repetição aparece na integração com OpenAI. O arquivo “src/vanna/integrations/openai/llm.py” possui lógica para extrair e interpretar chamadas de ferramentas tanto no fluxo de streaming quanto no fluxo normal. Em ambos os casos, há tratamento semelhante para converter argumentos JSON em dicionário e, em caso de erro ou formato inesperado, armazenar os dados em estruturas alternativas como “_raw” ou “args”. Isso indica uma oportunidade de extração para uma função utilitária compartilhada, evitando que a mesma regra de interpretação de argumentos seja mantida em mais de um ponto do código.

```
243
244  def _extract_tool_calls_from_message(self, message: Any) -> List[ToolCall]:
245      tool_calls: List[ToolCall] = []
246      raw_tool_calls = getattr(message, "tool_calls", None) or []
247      for tc in raw_tool_calls:
248          fn = getattr(tc, "function", None)
249          if not fn:
250              continue
251          args_raw = getattr(fn, "arguments", "{}")
252          try:
253              loaded = json.loads(args_raw)
254              if isinstance(loaded, dict):
255                  args_dict: Dict[str, Any] = loaded
256              else:
257                  args_dict = {"args": loaded}
258          except Exception:
259              args_dict = {"_raw": args_raw}
260          tool_calls.append(
261              ToolCall(
262                  id=getattr(tc, "id", "tool_call"),
263                  name=getattr(fn, "name", "tool"),
264                  arguments=args_dict,
265              )
266          )
267      return tool_calls
```

O trecho do adapter da OpenAI mostra o tratamento de argumentos de ferramentas durante o fluxo de streaming.

Portanto, a duplicação encontrada não deve ser classificada como um erro grave de arquitetura. Ela é parcialmente consequência do próprio padrão Adapter, já que cada provedor externo possui formato de requisição, resposta, streaming e tool calling diferentes. Entretanto, há trechos repetidos que poderiam ser movidos para uma camada utilitária ou para uma classe base abstrata, principalmente as rotinas de parsing de argumentos, validação comum de ferramentas, normalização de uso/tokens e montagem parcial de respostas.

Eixo C.1 - Padrões Criacionais

Singleton — Gerenciamento de Conexões com Banco de Vetores Status

O projeto não implementa Singleton para garantir que apenas uma conexão seja mantida com o banco de vetores (ChromaDB, Qdrant, etc.). Isso pode resultar em:

- Múltiplas conexões desnecessárias a APIs pagas
- Consumo excessivo de memória
- Overhead de latência em operações de embedding

Exemplo de problema em `src/vanna/integrations/chromadb/agent_memory.py`: Cada instância de `ChromaDB_AgentMemory` cria sua própria conexão, sem verificar se já existe uma instância ativa.

(https://github.com/vanna-ai/vanna/blob/main/src/vanna/integrations/chromadb/agent_memory.py)

```
def _get_client(self):
    """Get or create ChromaDB client."""
    if self._client is None:
        self._client = chromadb.PersistentClient(
            path=self.persist_directory,
            settings=Settings(anonymized_telemetry=False, allow_reset=True),
        )
    return self._client
```

Figura 4: criação do CHromaDB client

O mesmo objeto reutiliza o cliente, mas objetos diferentes criam clientes diferentes

Isso não é Singleton global, porque cada instância terá seu próprio `_client` e sua própria conexão, não existe gerenciamento centralizado entre instâncias

Recomenda-se introduzir uma `VectorStoreFactory` responsável por reutilizar clientes vetoriais compartilhados entre instâncias da aplicação, aplicando o padrão Singleton para reutilização de conexões, redução do consumo de memória e melhor escalabilidade.

```

class ChromaDBConnection:
    _instance = None

    @classmethod
    def get_instance(cls):
        if cls._instance is None:
            cls._instance = chromadb.Client(...)
        return cls._instance

```

Figura 5: aplicar singleton

Factory Pattern — Integração com Diferentes LLM Providers

A arquitetura do Agent usa Factory Method implícito através do sistema de injeção de dependências. Novos provedores de LLM podem ser adicionados sem modificar código existente:

- class OpenAILlmService(LlmService): ...
- class AnthropicLlmService(LlmService): ...
- class OllamaLlmService(LlmService): ...
- # src/vanna/integrations/openai.py class OpenAILlmService(LlmService):
- async def send_request(...) -> LlmResponse: ...
- #src/vanna/integrations/anthropic.py class AnthropicLlmService(LlmService):
- Async def send_request(...) -> LlmResponse: ...

- src/vanna/integrations/openai.py

```

class OpenAILlmService(LlmService):
    """OpenAI Chat Completions-backed LLM service.

    Args:
        model: OpenAI model name (e.g., "gpt-5").
        api_key: API key; falls back to env `OPENAI_API_KEY`.
        organization: Optional org; env `OPENAI_ORG` if unset.
        base_url: Optional custom base URL; env `OPENAI_BASE_URL` if unset.
        extra_client_kwargs: Extra kwargs forwarded to `openai.OpenAI()`.
    """

    def __init__(
        self,
        model: Optional[str] = None,
        api_key: Optional[str] = None,
        organization: Optional[str] = None,
        base_url: Optional[str] = None,
        **extra_client_kwargs: Any,
    ) -> None:

```

Figura 6: classe para LLMservice da OpenAI

- src/vanna/integrations/anthropic.py

```
class AnthropicLlmService(LlmService):
    extra_client_kwargs: Extra kwargs forwarded to `anthropic.Anthropic()`.
    """

    def __init__(
        self,
        model: Optional[str] = None,
        api_key: Optional[str] = None,
        base_url: Optional[str] = None,
        **extra_client_kwargs: Any,
    ) -> None:
        try:
```

Figura 7: class para LLMservice da Anthropic

Embora existam múltiplas implementações de LlmService, não existe uma Factory explícita centralizando a seleção do provedor. A escolha ocorre via injeção de dependências, funcionando como Factory implícita.

Recomenda-se criar uma LlmFactory para centralizar a criação dos provedores de IA:

```
class LlmFactory:
    @staticmethod
    def create(provider):
        match provider:
            case "openai":
                return OpenAILlmService()
            case "anthropic":
                return AnthropicLlmService()
            case "ollama":
                return OllamaLlmService()
            case _:
                raise ValueError(
                    "Provider não suportado"
                )
```

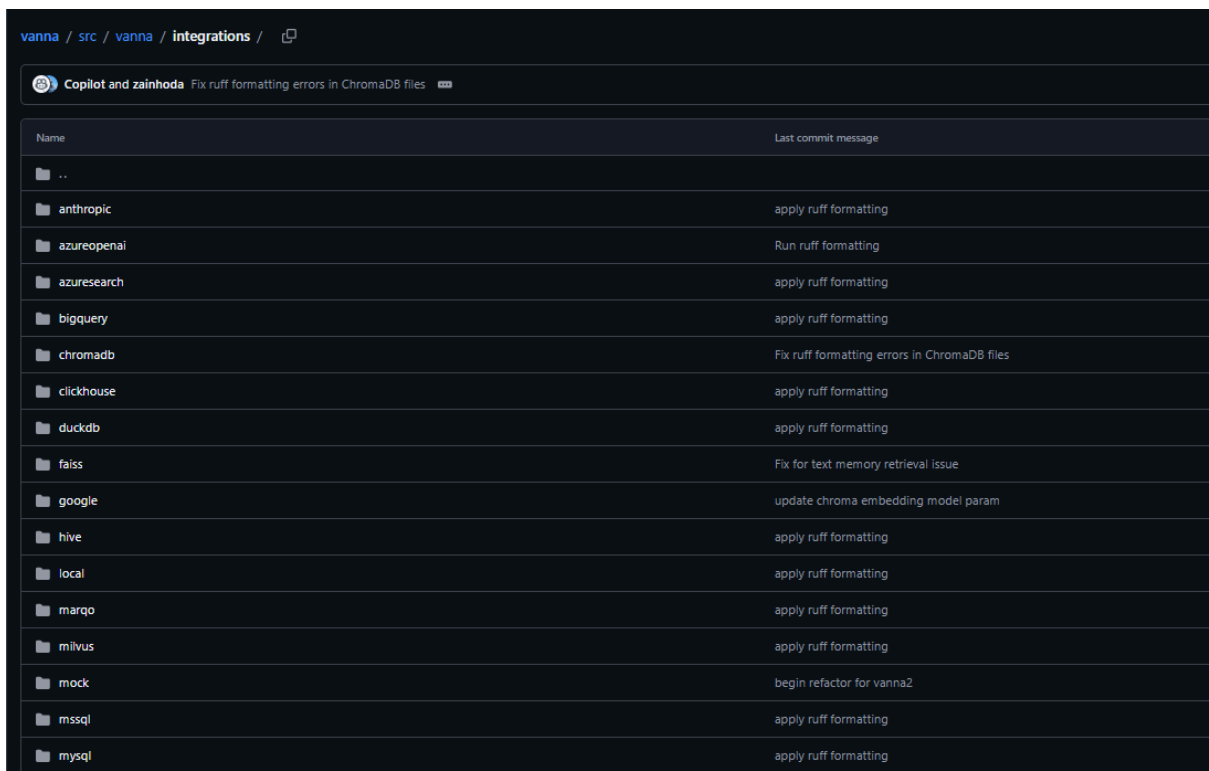
Figura 8: LlmFactory

Benefícios esperados:

- Redução de acoplamento entre agentes e provedores;
- Facilita inclusão de novos modelos de IA;
- Evita alterações em múltiplos arquivos;
- Maior aderência ao padrão GoF Factory Method.

Eixo C.2 - Padrões Estruturais

No repositório Vanna, há evidências fortes de uso do padrão Adapter, especialmente na camada de integração com provedores de LLM e bancos de dados. A Vanna possui uma pasta de integrações com diversos provedores e tecnologias, incluindo OpenAI, Anthropic, Google, entre outros. Essa organização indica que o projeto precisa lidar com múltiplas APIs externas, cada uma com interface própria, mas sem obrigar o restante do sistema a conhecer diretamente todos esses detalhes.



vanna / src / vanna / integrations	
Copilot and zainhoda Fix ruff formatting errors in ChromaDB files	
Name	Last commit message
..	
anthropic	apply ruff formatting
azureopenai	Run ruff formatting
azuresearch	apply ruff formatting
bigquery	apply ruff formatting
chromadb	Fix ruff formatting errors in ChromaDB files
clickhouse	apply ruff formatting
duckdb	apply ruff formatting
faiss	Fix for text memory retrieval issue
google	update chroma embedding model param
hive	apply ruff formatting
local	apply ruff formatting
marqo	apply ruff formatting
milvus	apply ruff formatting
mock	begin refactor for vanna2
mssql	apply ruff formatting
mysql	apply ruff formatting

A principal evidência estrutural está na interface “LlmService”, que define um contrato comum para os provedores de LLM. Independentemente de o modelo usado ser OpenAI, Anthropic ou Gemini, todos precisam se adaptar a métodos comuns como “send_request”, “stream_request” e “validate_tools” como demonstrado no Eixo B.3. Isso corresponde ao padrão Adapter, pois classes específicas convertem APIs externas diferentes para uma interface esperada pelo núcleo da aplicação.

Além disso, o projeto define modelos comuns como “LlmRequest”, “LlmResponse” e “LlmStreamChunk”. Esses objetos funcionam como uma saída padronizada: cada

provedor pode ter seu próprio formato de resposta, mas o restante da aplicação passa a consumir uma estrutura comum. Isso reduz o acoplamento e facilita a substituição de provedores de IA.

```
vanna / src / vanna / core / llm / models.py

Code Blame 61 lines (44 loc) · 1.92 KB

14
15 class LlmMessage(BaseModel):
16     """Message format for LLM communication."""
17
18     role: str = Field(description="Message role")
19     content: str = Field(description="Message content")
20     tool_calls: Optional[List[ToolCall]] = Field(default=None)
21     tool_call_id: Optional[str] = Field(default=None)
22
23
24 class LlmRequest(BaseModel):
25     """Request to LLM service."""
26
27     messages: List[LlmMessage] = Field(description="Messages to send")
28     tools: Optional[List[Any]] = Field(
29         default=None, description="Available tools"
30     ) # Will be ToolSchema but avoiding circular import
31     user: User = Field(description="User making the request")
32     stream: bool = Field(default=False, description="Whether to stream response")
33     temperature: float = Field(default=0.7, ge=0.0, le=2.0)
34     max_tokens: Optional[int] = Field(default=None, gt=0)
35     system_prompt: Optional[str] = Field(
36         default=None, description="System prompt for the LLM"
37     )
38     metadata: Dict[str, Any] = Field(default_factory=dict)
39
40
41 class LlmResponse(BaseModel):
42     """Response from LLM."""
43
44     content: Optional[str] = None
45     tool_calls: Optional[List[ToolCall]] = None
46     finish_reason: Optional[str] = None
47     usage: Optional[Dict[str, int]] = None
48     metadata: Dict[str, Any] = Field(default_factory=dict)
49
50     def is_tool_call(self) -> bool:
51         """Check if this response contains tool calls."""
52         return self.tool_calls is not None and len(self.tool_calls) > 0
53
54
55 class LlmStreamChunk(BaseModel):
56     """Streaming chunk from LLM."""
57
58     content: Optional[str] = None
59     tool_calls: Optional[List[ToolCall]] = None
60     finish_reason: Optional[str] = None
61     metadata: Dict[str, Any] = Field(default_factory=dict)
```

As implementações concretas reforçam essa interpretação. O OpenAILlmService adapta a API de Chat Completions da OpenAI para o contrato interno da Vanna; o AnthropicLlmService faz o mesmo com a API Messages da Anthropic; e o serviço do Gemini adapta a API do Google GenAI para o mesmo formato interno. Cada classe

conhece os detalhes do seu provedor, mas entrega ao núcleo da aplicação uma resposta padronizada.

Já no caso da hierarquia de agentes, a Vanna 2.0 se apresenta como uma arquitetura baseada em Agent, substituindo a abordagem anterior centrada em VannaBase. O Agent é responsável por orquestrar interações com LLM, execução de ferramentas e gerenciamento de conversa. Ele recebe dependências como “llm_service”, memória, filtros, hooks de ciclo de vida e provedores de observabilidade.

```
81
82  def __init__(
83      self,
84      llm_service: LlmService,
85      tool_registry: ToolRegistry,
86      user_resolver: UserResolver,
87      agent_memory: AgentMemory,
88      conversation_store: Optional[ConversationStore] = None,
89      config: AgentConfig = AgentConfig(),
90      system_prompt_builder: SystemPromptBuilder = DefaultSystemPromptBuilder(),
91      lifecycle_hooks: List[LifecycleHook] = [],
92      llm_middlewares: List[LlmMiddleware] = [],
93      workflow_handler: Optional[WorkflowHandler] = None,
94      error_recovery_strategy: Optional[ErrorRecoveryStrategy] = None,
95      context_enrichers: List[ToolContextEnricher] = [],
96      llm_context_enhancer: Optional[LlmContextEnhancer] = None,
97      conversation_filters: List[ConversationFilter] = [],
98      observability_provider: Optional[ObservabilityProvider] = None,
99      audit_logger: Optional[AuditLogger] = None,
100  ):
101      self.llm_service = llm_service
102      self.tool_registry = tool_registry
103      self.user_resolver = user_resolver
104      self.agent_memory = agent_memory
```

Entretanto, a estrutura não parece apostar em uma hierarquia profunda de subclasses de agentes. A pasta src/vanna/agents aparece praticamente como um pacote reservado, sem implementações concretas exportadas. Isso mostra que o projeto provavelmente prefere em vez de criar vários tipos de agentes por subclasses, ele monta o comportamento do Agent por meio de serviços, ferramentas e registries.

Essa decisão é positiva do ponto de vista arquitetural, pois evita uma hierarquia rígida e difícil de manter. A extensibilidade ocorre por meio de componentes plugáveis, especialmente ferramentas. O projeto define uma abstração Tool, com métodos específicos, e um “ToolRegistry”, responsável por registrar ferramentas, validar permissões e executar ferramentas disponíveis ao usuário.

```

36     @abstractmethod
37     def get_args_schema(self) -> Type[T]:
38         """Return the Pydantic model for arguments."""
39         pass
40
41     @abstractmethod
42     async def execute(self, context: ToolContext, args: T) -> ToolResult:
43         """Execute the tool with validated arguments.
44
45         Args:
46             context: Execution context containing user, conversation_id, and request_id
47             args: Validated tool arguments
48
49         Returns:
50             ToolResult with success status, result for LLM, and optional UI component
51         """
52         pass
53
54     def get_schema(self) -> ToolSchema:
55         """Generate tool schema for LLM."""
56         from typing import Any, cast
57
58         args_model = self.get_args_schema()
59         # Get the schema - args_model should be a Pydantic model class
60         schema = (
61             cast(Any, args_model).model_json_schema()
62             if hasattr(args_model, "model_json_schema")
63             else {}
64         )
65         return ToolSchema(
66             name=self.name,
67             description=self.description,
68             parameters=schema,
69             access_groups=self.access_groups,
70         )

```

C.3 Padrões Comportamentais

O fluxo de raciocínio da IA é controlado, especificamente perguntando se existe uma "corrente" de processamento (*Chain of Responsibility*) ou apenas um "emaranhado" de condicionais *if/else*. No caso do **Vanna Legacy**, antes da refatoração para o "Vanna2", infelizmente, a resposta é o emaranhado de *if/else*. A orquestração principal de todo o fluxo (receber pergunta -> buscar RAG -> gerar SQL -> rodar SQL -> gerar código de gráfico -> plotar gráfico) é feita de forma estritamente processual em seu código legacy.

- (a) Evidência
 - Link: Vá para <https://github.com/vanna-ai/vanna/blob/main/src/vanna/legacy/base/base.py> (linhas 1692 - 1806, função `def ask`)

```

1692     def ask(
1725         if question is None:
1726             question = input("Enter a question: ")
1727
1728         try:
1729             sql = self.generate_sql(
1730                 question=question, allow_llm_to_see_data=allow_llm_to_see_data
1731             )
1732         except Exception as e:
1733             print(e)
1734             return None, None, None
1735
1736         if print_results:
1737             try:
1738                 from IPython.display import Code, display
1739
1740                 display(Code(sql))
1741             except Exception as e:
1742                 print(sql)
1743
1744         if self.run_sql_is_set is False:
1745             print("If you want to run the SQL query, connect to a database first.")
1746
1747         if print_results:
1748             return None
1749         else:
1750             return sql, None, None
1751
1752         try:
1753             df = self.run_sql(sql)
1754
1755             if print_results:
1756                 try:
1757                     display = __import__(
1758                         "IPython.display", fromList=["display"]
1759                     ).display
1760                     display(df)
1761                 except Exception as e:
1762                     print(df)
1763
1764             if len(df) > 0 and auto_train:
1765                 self.add_question_sql(question=question, sql=sql)
1766             # Only generate plotly code if visualize is True
1767             if visualize:
1768                 try:

```

Referente a versão legacy, o fluxo de processamento não utiliza padrões comportamentais elegantes como *Chain of Responsibility* ou *Command*. A lógica de orquestração está *hardcoded* (fixada) em um fluxo sequencial rígido dentro de um único método. A execução de cada etapa é condicionada por *flags* que vêm dos parâmetros da função. Isso torna o código engessado: se precisarmos adicionar uma nova etapa no meio do caminho (ex: "Validar segurança do SQL gerado para evitar *Drop Table*"), teremos que modificar diretamente a classe base, adicionando mais condicionais e violando o Princípio Aberto/Fechado (*Open/Closed Principle*) do SOLID.

Na versão LEGACY a classificação de Risco é ALTA. A dependência de um fluxo processual rígido aumenta a complexidade ciclomática com o tempo. Isso dificulta a manutenção, a testabilidade unitária de etapas isoladas e a criação de *middlewares* personalizados pela comunidade.

O código da versão mais atualizada o “**Vanna2**” revela que o Vanna abandonou a orquestração processual (o antigo emaranhado de `if/else`) e passou a usar um padrão comportamental de **Agentic Loop** (Loop Agêntico), terceirizando a decisão de fluxo para a própria Inteligência Artificial.

- (a) Evidência:

- **Print 1 (O Loop de Raciocínio):** Arquivo `agent.py`, vá para o método `_send_message`. Print do trecho (fonte <https://github.com/vanna-ai/vanna/blob/main/src/vanna/core/agent/agent.py>)

```
645
646         while tool_iterations < self.config.max_tool_iterations:
647             if self.config.include_thinking_indicators and tool_iterations == 0:
648                 # TODO: Yield thinking indicator
649                 pass
650
651             # Get LLM response
652             if self.config.stream_responses:
653                 response = await self._handle_streaming_response(request)
654             else:
655                 response = await self._send_llm_request(request)
656
657             # Handle tool calls
658             if response.is_tool_call():
659                 tool_iterations += 1
660
```

- while `tool_iterations < self.config.max_tool_iterations`: (Isso prova que o fluxo agora é um loop iterativo).
- **Print 2 (A Execução Cega):** Mais abaixo, dentro do bloco `for i, tool_call in enumerate(response.tool_calls or []):`, fonte (<https://github.com/vanna-ai/vanna/blob/main/src/vanna/core/agent/agent.py>)

```

822         "tool": tool_call.name,
823         "arg_count": len(tool_call.arguments),
824     },
825 )
826
827     result = await self.tool_registry.execute(tool_call, context)
828
829     if self.observability_provider and tool_exec_span:
830         tool_exec_span.set_attribute("success", result.success)
831         if not result.success:
832             tool_exec_span.set_attribute(
833                 "error", result.error or "unknown"
834             )
835         await self.observability_provider.end_span(tool_exec_span)
836         if tool_exec_span.duration_ms():
837             await self.observability_provider.record_metric(
838                 "agent.tool.duration",
839                 tool_exec_span.duration_ms() or 0,
840                 "ms",
841                 tags={
842                     "tool": tool_call.name,
843                     "success": str(result.success),
844                 },
845             )

```

`result = await self.tool_registry.execute(tool_call, context)` (Isso prova onde a ação ocorre).

- Print 3

https://github.com/vanna-ai/vanna/blob/main/src/vanna/tools/run_sql.py

```

45     @property
46     def description(self) -> str:
47         return (
48             self._custom_description
49             if self._custom_description
50             else "Execute SQL queries against the configured database"
51         )
52
53     def get_args_schema(self) -> Type[RunSqlToolArgs]:
54         return RunSqlToolArgs
55
56     async def execute(self, context: ToolContext, args: RunSqlToolArgs) -> ToolResult:
57         """Execute a SQL query using the injected SqlRunner."""
58         try:
59             # Use the injected SqlRunner to execute the query
60             df = await self.sql_runner.run_sql(args, context)
61
62             # Determine query type
63             query_type = args.sql.strip().upper().split()[0]
64
65             if query_type == "SELECT":
66                 # Handle SELECT queries with results
67                 if df.empty:
68                     result = "Query executed successfully. No rows returned."
69                     ui_component = UiComponent(

```

-

executa a query cegamente.

- **Análise Crítica** : No **Vanna2**, o controle de fluxo mudou drasticamente. Em vez de uma orquestração rígida, o Agente adota um laço (`while`) que confia

totalmente na capacidade do LLM de escolher as ferramentas. O framework até possui `lifecycle_hooks` (ganchos como `before_tool`), mas eles funcionam como meros observadores ou injetores de contexto, não como uma verdadeira *Chain of Responsibility* feita para barrar ou sanitizar lógicas de negócio estruturais. A arquitetura delega a responsabilidade comportamental para o LLM, executando as `tool_calls` de maneira "cega" e direta através do `tool_registry.execute()`.

- **Classificação de Risco para o Vanna 2 é Alto.** A dependência exclusiva do LLM para o controle de fluxo (sem uma corrente de validação comportamental forte no código do Agente) cria uma vulnerabilidade clássica de IA. Se o modelo tiver uma alucinação e decidir invocar uma ferramenta com argumentos perigosos, a linha `await self.tool_registry.execute` rodará o comando no banco de dados imediatamente.

4.1 Refatoração Conceitual (codigo sera disponibilizado no repo apos organizarem)

O nosso plano de resgate agora será aplicar o **Chain of Responsibility** para criar um "Escudo" em volta da execução das *Tools* do novo Vanna.

- **Código Violado:** O mecanismo de invocação das ferramentas do agente (que confia cegamente na saída do LLM para rodar uma *Tool*).
- **Padrão Aplicado: Chain of Responsibility** (Implementado como *Middlewares/Interceptors* de Execução).
- **Justificativa Arquitetural:** > A equipe de auditoria identificou que o repositório já utiliza o padrão *Chain of Responsibility/Middleware* na camada de comunicação com as APIs de IA (conforme evidenciado em <https://github.com/vanna-ai/vanna/tree/main/src/vanna/core/middleware> com a classe `LlmMiddleware`). No entanto, **houve uma falha de design arquitetural ao não estender esse mesmo padrão para a camada de Execução de Ferramentas (`ToolRegistry` e `Agent`)**. A nossa proposta corrige essa dívida técnica criando um `ExecutionMiddleware`, aplicando uma barreira de segurança onde ela é mais crítica: entre a decisão/conclusão da IA e a execução real no banco de dados.

Se o Vanna adotasse o nosso padrão da 4.1, o fluxo seria:

1. O Agente pede para rodar um SQL.
2. O pedido entra na **Corrente de Responsabilidade** (nosso `ToolExecutionRequest`).
3. O `SQLSanitizationMiddleware` (nosso elo da corrente) lê o `args.sql`.
4. Se ele ver a palavra `DROP` ou `DELETE`, a corrente **quebra a execução ali mesmo** e devolve um erro para a IA.
5. O método `await self.sql_runner.run_sql(args, context)` do `run_sql.py` nunca chega a ser chamado.

•

```

1  # 1. O Contexto do Pedido da Ferramenta
2  class ToolExecutionRequest:
3      def __init__(self, tool_name, arguments, user_context):
4          self.tool_name = tool_name
5          self.arguments = arguments
6          self.user_context = user_context
7          self.is_blocked = False
8
9  # 2. A Corrente de Responsabilidade (Handler Base)
10 class ExecutionMiddleware:
11     def __init__(self, next_middleware=None):
12         self.next_middleware = next_middleware
13
14     def handle(self, request: ToolExecutionRequest):
15         if not request.is_blocked and self.next_middleware:
16             return self.next_middleware.handle(request)
17
18 # 3. Os Elos da Corrente (Validações de Segurança e Negócio)
19 class RateLimitMiddleware(ExecutionMiddleware):
20     def handle(self, request: ToolExecutionRequest):
21         if check_quota(request.user_context) > LIMIT:
22             request.is_blocked = True
23             return "Erro: Limite de uso excedido."
24         return super().handle(request)
25
26 class SQLSanitizationMiddleware(ExecutionMiddleware):
27     def handle(self, request: ToolExecutionRequest):
28         if request.tool_name == "run_sql":
29             query = request.arguments.get('sql', '').upper()
30             if "DROP" in query or "DELETE" in query:
31                 request.is_blocked = True
32                 return "Erro: AI tentou rodar query destrutiva!"
33             return super().handle(request)
34
35 class FinalExecutionMiddleware(ExecutionMiddleware):
36     def handle(self, request: ToolExecutionRequest):
37         # Se chegou até aqui, a IA está autorizada a rodar a ferramenta
38         return tool_registry.execute(request.tool_name, request.arguments)
39
40 # 4. Uso na Arquitetura Vanna 2.0
41 # Agora o Agente não ataca o banco diretamente. Ele passa pela corrente!
42 pipeline_seguranca = RateLimitMiddleware(SQLSanitizationMiddleware(FinalExecutionMiddleware()))
43
44 # Quando a IA pede para rodar algo:
45 resultado = pipeline_seguranca.handle(ToolExecutionRequest(
46     tool_name="run_sql",
47     arguments={"sql": "DROP TABLE usuarios;"},
48     user_context=current_user
49 ))

```

•

4.2 Diferencial: Plano de Resgate Sugerido

4.2.1 Refatoração Conceitual – Chain of Responsibility na Execução de Ferramentas

A auditoria identificou que o agente do **Vanna 2.0** executa chamadas de ferramentas de forma cega (sem camada de validação entre a decisão do LLM e a execução no banco de dados). O padrão **Chain of Responsibility** foi proposto para criar uma corrente de middlewares de execução (*ExecutionMiddleware*) que intercepta cada *tool_call* antes do `tool_registry.execute()`. O código de implementação encontra-se no repositório da equipe. O fluxo resultante é:

Etapa	Componente	Ação
1	Agent._send_message()	Recebe tool_call do LLM
2	ToolExecutionChain	Inicia a corrente de middlewares
3	SQLSanitizationMiddleware	Bloqueia DROP/DELETE não autorizados
4	RateLimitMiddleware	Controla frequência de chamadas a APIs pagas
5	LoggingMiddleware	Registra auditoria de cada execução
6	tool_registry.execute()	Executa apenas se toda a corrente aprovar

4.2.2 Roadmap de Maturidade – Alinhamento ao Nível G do MPS.BR

O MPS.BR (Melhoria de Processo do Software Brasileiro) define sete níveis de maturidade (G A). O **Nível G**, ponto de entrada, exige a institucionalização de dois processos fundamentais: **Gerência de Projetos (GPR)** e **Gerência de Requisitos (GRE)**. A análise do repositório *vanna-ai/vanna* evidenciou que o projeto opera de maneira informal, sem processos definidos, configurando estágio equivalente ao pré-Nível G do MPS.BR. As três ações prioritárias abaixo formam o roadmap para alcançar o Nível G em até 90 dias.

AÇÃO 1 — Institucionalizar a Gerência de Projetos (GPR) via GitHub Projects

Processo MPS.BR alvo: GPR (Gerência de Projetos e Riscos). **Prazo sugerido:** Semanas 1–3 (implementação imediata). **Prioridade:** CRÍTICA.

■ EVIDÊNCIA – Gráfico de contribuições – GitHubInsights

Análise do histórico de commits (ago/2023, mar/2024, nov/2025) evidencia picos concentrados de atividade seguidos de longos períodos de baixo movimento. O principal desenvolvedor concentra aproximadamente 6×

Contributors

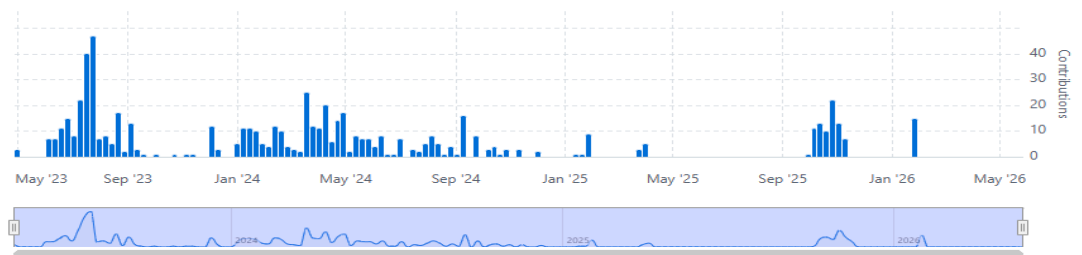
Contributions per week to main, excluding merge commits

Period: Custom range

Contributions: Commits

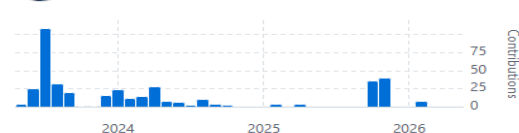
Commits over time

Weekly from 30 de abr. de 2023 to 23 de mai. de 2026



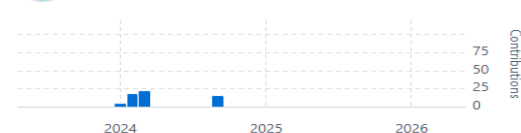
zainhoda
399 commits 99.620 ++ 49.449 --

#1



andreped
60 commits 711 ++ 229 --

#2



A auditoria (Eixo A.3) detectou ritmo de desenvolvimento irregular com picos de *crunch* e ausência de cadência previsível. Além disso, identificou-se concentração excessiva de contribuições em um único mantenedor (~6× mais commits que o segundo colaborador), configurando o padrão de *herói único* — risco

classificado como ALTO no MPS.BR.

Ações concretas:

- Criar um **GitHub Project Board** com colunas: Backlog In Progress Review Done, mapeando cada issue a um milestone mensal com data de entrega definida.
- Adotar o padrão **Conventional Commits** (feat, fix, docs, chore) para rastreabilidade automatizada via CHANGELOG e releases semanticamente versionadas.
- Estabelecer **obrigatoriedade de aprovação de pelo menos 2 revisores** em Pull Requests que afetem módulos críticos (core/agent, core/recovery, legacy/base).
- Implementar **rotação de mantenedores**: designar 2 co-maintainers com acesso de merge, reduzindo o risco de dependência de herói único.
- Criar um **Registro de Riscos** no wiki do repositório, documentando formalmente os riscos identificados (timeouts de API, mudanças de versão de LLM, vendor lock-in) com impacto, probabilidade e estratégia de mitigação.

Resultado esperado (GPR): Planejamento documentado, estimativas de esforço registradas e monitoramento explícito de desvios — requisitos mínimos do processo GPR no Nível G.

AÇÃO 2 — Implementar Gerência de Requisitos (GRE) com Template Estruturado de Issues

Processo MPS.BR alvo: GRE (Gerência de Requisitos). **Prazo sugerido:** Semanas 2–5. **Prioridade:**

ALTA.

A auditoria (Eixo A.1) demonstrou que decisões técnicas relevantes são tomadas informalmente nos comentários de issues e PRs, sem RFCs ou documentação estruturada. A issue #659 evidenciou mudança de escopo não documentada durante a correção do PGVector, ampliando o escopo de um bug isolado para mudanças arquiteturais. Essa rastreabilidade deficiente viola diretamente os critérios da GRE.

■ EVIDÊNCIA – Issue #659 – PGVector networking

A issue foi aberta reportando erro pontual (documentation_collection inexistente). Ao longo da discussão, o escopo expandiu para cobrir inconsistências assíncronas, falhas em embeddings e limitações arquiteturais —sem documentação formal da mudança de escopo. PR #660 consolidou correções sem rastreamento explícito de requisitos. Link: <https://github.com/vanna-ai/vanna/issues/659>

PG Vector not working #659 Closed

At least something like that should work.

IsaacDalmacio on Oct 11, 2024

Hi @andredp, I am trying to use pgvector as a custom vector database and I have copied the pgvector to my project. But when I start I get the error:

```
./pymc/versions/2.12.6/lib/python3.12/site-packages/langchain_postgres/vectorstores.py", line 105, in
_get_embedding_collection_store
from pgvector.sqlalchemy import Vector # type: ignore
ModuleNotFoundError: No module named 'pgvector.sqlalchemy': 'pgvector' is not a package
```

I tried to run in python 3.12 and 3.11, changed pgvector and langchain_postgres versions, but none worked.

Which version of python are you using? Do you have any other suggestion?

andredp on Oct 11, 2024

@IsaacDalmacio I think we should wait till PR #660 is merged, as the code in the main branch for the pgvector implementation is broken.

Then we can try to see how to fix the issue of yours, if it is still an issue after merge.

zainhoda linked a pull request that will close this issue [PGvector fixes #660](#) on Oct 23, 2024

andredp on Oct 25, 2024

@andredp, @IsaacDalmacio A new release of `pgvector` was just released (see [here](#)), which should have resolved the original issue and maybe the one @IsaacDalmacio is having.

Could you try to upgrade to the latest version and test if this new version works well with you? ;)

Assignees: No one assigned

Labels: bug

Type: No type

Fields: [View feedback](#)

Projects: No projects

Milestones: No milestone

Relationships: [PGvector fixes #660](#)

Notifications: [Subscribe](#)

Participants: IsaacDalmacio, andredp, zainhoda

Ações concretas:

- Criar **Issue Templates** no GitHub (.github/ISSUE_TEMPLATE/) com campos obrigatórios: Descrição do Requisito, Critério de Aceitação, Impacto Arquitetural e Módulos Afetados.
- Adotar o formato **User Story** para novas features: "Como [papel], quero [funcionalidade] para [benefício]", com critérios de aceitação mensuráveis.
- Implementar **labels de rastreabilidade** obrigatórias: req:functional, req:non-functional, arch:breaking, risk:high/medium/low.
- Criar um documento **ARCHITECTURE.md** na raiz do repositório registrando as decisões arquiteturais (ADRs – Architecture Decision Records) com contexto, decisão tomada e consequências.

- Configurar **branch protection rules** exigindo que todo PR referencia pelo menos uma issue rastreável antes do merge.

Resultado esperado (GRE): Requisitos consistentemente documentados, com bidirecionalidade entre requisitos e implementações — critério central da GRE no Nível G.

AÇÃO 3 — Elevar a Qualidade do Produto (QPR) via Pipeline de CI/CD e Testes Automatizados

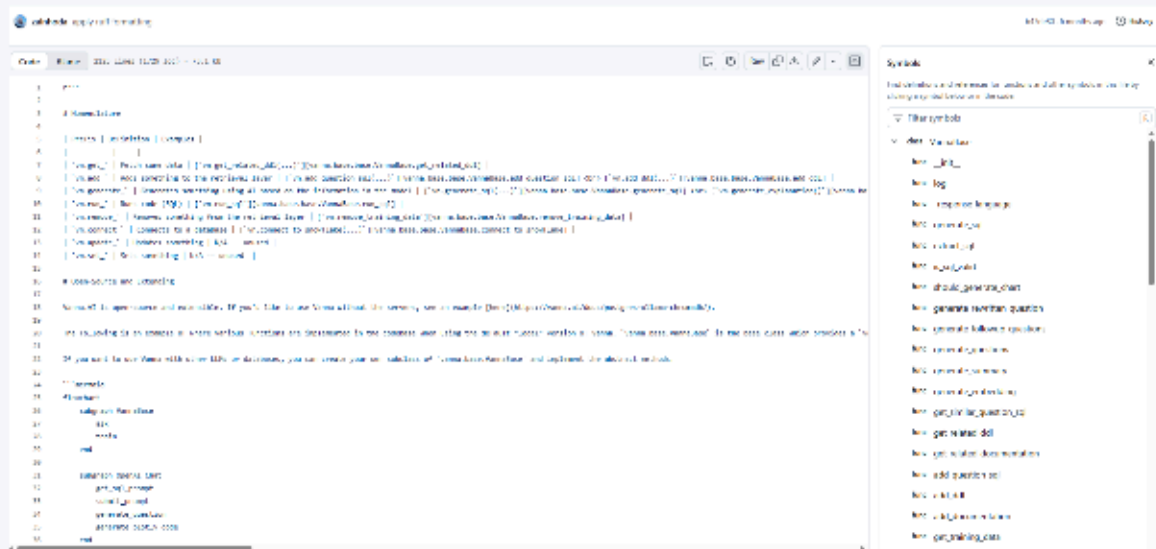
Processo MPS.BR alvo: Qualidade do Produto e do Processo (suporte a GPR + GRE).
Prazo sugerido:

Semanas 3–8. **Prioridade:** ALTA.

A auditoria (Eixos A.1, B.2, C.3) evidenciou que funcionalidades foram integradas sem cobertura de testes adequada (caso PGVector: *'hadn't been tested sufficiently'*), que a classe VannaBase (2.125 linhas, 65 métodos) não possui testes unitários isolados, e que o agente executa `tool_calls` sem validação comportamental — criando vulnerabilidade direta de injeção SQL via alucinação do LLM.

■ EVIDÊNCIA – VannaBase – God Class sem isolamento de testes

Classe identificada em: `src/vanna/legacy/base/base.py` (2.125 linhas, 65 métodos). Mescla 7 responsabilidades distintas (geração de SQL, conexão com BD, embeddings, orquestração, LLM, prompts, avaliação), impossibilitando testes unitários isolados. Link: <https://github.com/vanna-ai/vanna/blob/main/src/vanna/legacy/base/base.py>



Ações concretas:

- Configurar **GitHub Actions** com pipeline obrigatório em todo PR: lint (flake8/ruff) testes unitários (pytest) cobertura mínima de 70% (coverage.py).
- Implementar **testes de contrato** para cada integração de LLM: garantir que OpenAILlmService, AnthropicLlmService e OllamaLlmService retornam o mesmo schema LlmResponse, prevenindo regressões silenciosas em provedores.

- Criar **testes de segurança automatizados** para o ExecutionMiddleware proposto na 4.1: casos de teste com payloads maliciosos (DROP TABLE, DELETE FROM) verificando que a corrente bloqueia a execução antes do banco de dados.
- Introduzir **análise estática de segurança (SAST)** com Bandit (Python) integrada ao pipeline de CI, detectando automaticamente padrões de SQL injection e execução insegura.
- Estabelecer **Definition of Done (DoD)** formal para o projeto: toda feature só é considerada pronta quando possui testes automatizados, documentação atualizada e aprovação de pelo menos um revisor técnico.

Resultado esperado: Pipeline de qualidade automatizado que previne regressões, reduz dívida técnica acumulada e aumenta a confiança na evolução do sistema — base para os processos de Verificação e Validação exigidos nos níveis superiores do MPS.BR.

Síntese do Roadmap de Maturidade – 90 dias para o Nível G

Ação	Processo MPS.BR	Prazo	Prioridade	Impacto
Ação 1 GitHub Projects + Rotação de Mantenedores	GPR – Gerência de Projetos e Riscos	Semanas 1–3	CRÍTICA	Eliminar o código heróico e criar cadência previsível
Ação 2 Issue Templates + ADRs + GRE	GRE – Gerência de Requisitos	Semanas 2–5	ALTA	Rastreabilidade bidirecional entre req. e implementação
Ação 3 CI/CD + Testes + SAST + DoD	QPR – Qualidade do Produto	Semanas 3–8	ALTA	Prevenir regressões e bloqueia vulnerabilidades de segurança

Nota: A execução sequencial das três ações, no prazo de 90 dias, posiciona o projeto para submissão à avaliação formal do Nível G MPS.BR. Os processos GPR e GRE são pré-requisito obrigatório antes de qualquer tentativa de alcançar o Nível F (que adiciona Gerência de Configuração e Medição).